CROWFIT

CONTEXTO MESTRE DO PROJETO

Base permanente para prompts de desenvolvimento, correção e evolução do aplicativo.

OBJETIVO DESTE DOCUMENTO

Fornecer a qualquer IA de programação o contexto do Crowfit: propósito, funcionalidades, arquitetura, dados, autenticação, decisões de produto, problemas conhecidos e regras para alterar o projeto com segurança.

VERSÃO	DATA DE REFERÊNCIA	FONTE TÉCNICA
1.0	10 de novembro de 2026	Repositório público GitHub - branch main

Identificação do projeto

Crowfit é um aplicativo mobile-first de treino físico sem equipamentos de academia, com mascote corvo, trilha de evolução, sessões guiadas, pontos e ofensiva.

Repositório GitHub

Aplicativo publicado

REGRA DE PRECEDÊNCIA

Este PDF é contexto de referência, não substitui o código atual. Antes de cada tarefa, a IA deve conferir a branch e os arquivos reais do repositório. Se o código atual divergir deste documento, informar a divergência e usar o código mais recente como fonte de verdade, sem ignorar as decisões explícitas do usuário.

Uso recomendado: anexar este PDF junto de cada prompt. No prompt de cada tarefa, descrever apenas a mudança desejada, o resultado esperado e eventuais referências visuais.

01 / PRODUTO

Visão do Crowfit

O que o aplicativo é, para quem foi pensado e como deve se comportar.

Proposta central

O Crowfit ajuda a pessoa a treinar em casa, no quarto, no parque ou onde estiver, sem depender de equipamentos de academia. O aplicativo transforma a consistência em uma experiência de progressão: cada dia tem uma unidade de treino ou recuperação, o usuário acompanha o avanço numa trilha e recebe pontos e evolução de nível.

Princípios de produto

- Mobile-first: a experiência principal deve funcionar muito bem em telas pequenas e por toque.
- Baixa fricção: instruções objetivas, um exercício por vez e poucos passos para começar a treinar.
- Progressão gradual: o treino evolui por etapas; não deve aumentar a carga de maneira brusca.
- Continuidade: o usuário não deve perder histórico, pontos, ofensiva ou progresso ao editar o perfil, trocar de tela ou atualizar o app.
- Clareza: descanso, exercício concluído, exercício pendente, treino liberado e treino futuro devem ser visualmente distintos.
- Segurança: incluir orientação para interromper o exercício em caso de dor ou tontura; não apresentar o app como substituto de avaliação profissional.

Onboarding atual

O primeiro cadastro de treino pergunta, nesta ordem: nome, idade, peso em kg, altura em cm, experiência com exercícios e quantos dias por semana deseja treinar (3, 4, 5 ou 6). A validação do código aceita nome de 1 a 40 caracteres, idade inteira de 12 a 90, peso de 30 a 250 kg e altura de 120 a 230 cm.

PONTO IMPORTANTE

O onboarding de treino é diferente do cadastro/autenticação da conta. Criar uma conta Supabase não significa que o perfil de treino daquela conta já exista. O aplicativo precisa verificar o estado de treino associado ao usuário antes de decidir entre mostrar o onboarding ou abrir a trilha.

Identidade visual atual

O repositório atual usa um visual escuro como base, com fundo quase preto (#080808), cartões cinza-escuros, texto claro e vermelho Crowfit (#D92332) como cor de destaque. Os títulos usam Fredoka e o restante da interface usa Nunito. O mascote é um corvo preto presente no app, nos ícones e no logotipo.

Existe também uma direção visual solicitada anteriormente para uma tela de apresentação/onboarding: fundo off-white, logotipo Crowfit, mascote PNG original sem redesenho ou recoloração, enfeites discretos em CSS/SVG, título “Treine, evolua e seja mais forte!” e CTA vermelho “Vamos começar”. Tratar isso como referência visual anterior, não como estado já implementado; confirmar antes de aplicar caso conflite com o tema escuro atual.

02 / EXPERIÊNCIA DE TREINO

Mecânicas e regras do aplicativo

As regras abaixo devem ser preservadas ao mexer em telas, perfil, persistência ou sincronização.

Trilha de progressão

A trilha contínua é implementada principalmente por program.js e trail.js. O programa organiza exercícios por grupos musculares e recuperação, com seções de progressão de aproximadamente três meses cada. O treino de cada dia é derivado da data de início, do calendário, do grupo muscular do dia e do nível-base registrado para a seção.

- Grupos codificados no programa: recuperação (R), peito (P), braços (B), abdômen (A), pernas (L), costas (C) e ombros (O).
- A biblioteca de exercícios fica em CFP.LIB, identificada por IDs estáveis. O ID deve ser preservado mesmo quando o nome ou a descrição de um exercício mudar, para não quebrar o histórico de conclusão.
- A trilha libera cada unidade na data correspondente; não deve permitir antecipar unidades futuras.
- O nível-base de uma seção fica congelado em S.trail.lv para evitar que treinos passados mudem retroativamente quando o nível do usuário se ajustar.
- Dentro de uma seção, o volume aumenta gradualmente ao longo das semanas; ao iniciar uma nova seção, o nível de referência sobe conforme a regra do programa.

Execução guiada

A experiência de treino apresenta um exercício por vez, com número de séries, repetições ou tempo, instruções de execução e descanso. Exercícios cronometrados precisam respeitar estados de início, pausa, retomada e conclusão do tempo. O descanso pode ser pulado. O usuário pode voltar à trilha sem perder os exercícios já concluídos.

No executor da trilha (trail.js), o progresso persistido é o estado de conclusão de cada exercício, identificado pela chave da unidade e pelo ID do exercício. O estado temporário da série/cronômetro é mantido no executor enquanto aquela sessão está aberta; não presumir que todo o estado de uma série incompleta seja recuperado após fechar o app.

Pontos, níveis e ofensiva

- A interface calcula o nível a partir dos pontos: um novo nível a cada 200 pontos.
- Na trilha, ao completar uma unidade, a regra atual concede 10 pontos por exercício da unidade e 20 pontos de bônus, apenas uma vez para aquela unidade.
- A unidade concluída marca o dia como treinado e atualiza a ofensiva por meio da lógica existente no core.js.
- O core.js também contém lógica de sessões com 10 pontos por exercício concluído e bônus de 20 pontos por sessão completa. Não misturar silenciosamente as regras da trilha com as regras do executor de sessões legado.
- Ajustes de dificuldade usam o retorno do usuário: “Foi difícil”, “No ponto” ou “Foi fácil”; três treinos fáceis consecutivos são necessários para elevar a dificuldade na lógica de sessões.

ATENÇÃO AOS DOIS MODELOS

O código mantém lógica de estado/sessões e plano semanal em core.js, enquanto program.js + trail.js montam a trilha progressiva. Antes de sincronizar ou refatorar, mapear exatamente quais telas e fluxos usam cada mecanismo. Não presumir que um formato de sessão representa todo o progresso da trilha.

03 / DADOS

O que é salvo e onde

Mapa do estado atual e conceitos que precisam ser preservados.

Estado principal em core.js

CAMPO	SIGNIFICADO
version	Versão do formato do estado local.
profile	Perfil de treino: nome, idade, peso, altura, experiência e dias por semana.
points	Pontuação total acumulada.
streak / best / last	Ofensiva atual, melhor ofensiva e última data de treino.
done	Datas marcadas como dias treinados.
rewards	Chaves de recompensas já concedidas; evita duplicar pontos no mesmo dispositivo/estado.
history / planHistory	Histórico de sessões e histórico de plano.
adjust / easyStreak	Ajuste de dificuldade e sequência de retornos “fácil”.
active	Sessão de treino em andamento no modelo de sessões do core.
trail	Progresso da trilha: data de início, exercícios concluídos por unidade e nível-base por seção.

Estrutura de uma sessão no core

Uma sessão pode conter ID, data, hora de início e fim, tipo, status, duração, feedback, pontuação, indicação de sessão extra e lista de exercícios. Cada exercício pode guardar ID, nome, tipo (repetições ou tempo), alvo, status, índice da série, descanso, cronômetro e séries concluídas. Ao mapear esses dados para um banco, respeitar nomes, tipos, datas e estados; não descartar campos desconhecidos sem uma migração explícita.

Estrutura específica da trilha

S.trail tem a forma conceitual { start, ex, lv }. “start” é a data de início; “ex” armazena exercícios concluídos, por exemplo na unidade s1u4; “lv” guarda o nível-base de cada seção. O restante da programação é calculado a partir da data e de program.js. Os IDs dos exercícios e as chaves de unidade são parte da compatibilidade dos dados.

Backup local atual

- Exporta um arquivo JSON com o estado do Crowfit (nome padrão crowfit-backup-AAAA-MM-DD.json).
- A importação valida o conteúdo antes de substituir o estado e pede confirmação ao usuário.
- Antes da importação, tenta guardar uma cópia do estado atual na chave corvofit-v1:antes-importar.
- O importador limita o arquivo a 1 MB. Se os dados estiverem corrompidos, a tela de recuperação preserva uma cópia quando possível.
- Refazer o perfil mantém pontos, histórico, ofensiva e trilha; apenas limpa S.profile para refazer o onboarding.

REGRA DE PRESERVAÇÃO

Nunca limpar localStorage, reiniciar estado ou sobrescrever backup como atalho para corrigir um problema. Antes de qualquer migração, criar uma cópia verificável e definir como reverter.

04 / ARQUITETURA

Mapa dos arquivos do repositório

Aplicação web simples: HTML, CSS e JavaScript puro, sem etapa de build conhecida no snapshot consultado.

ARQUIVO	RESPONSABILIDADE / CUIDADOS
index.html	Tela principal, estilos globais inline, definição de exercícios/sessões, onboarding, home, treino, backup e inicialização. Hoje carrega o estado local e decide entre onboarding e home.
core.js	Lógica pura: validação do perfil/estado, plano semanal legado, sessões, timers, pontos, feedback, ofensiva, histórico e backup helpers.
program.js	Biblioteca de exercícios, grupos musculares, escadas/variações e montagem dinâmica do treino por seção, semana e dia.
trail.js	Trilha contínua, unidades por data, conclusão de exercícios, liberação diária e pontuação da unidade.
auth.js	Cliente Supabase Auth, estado de sessão, login/cadastro por e-mail, OAuth Google, recuperação/troca de senha e logout.
auth-ui.js / auth.css	Overlay de login, cadastro, Google, recuperação, mensagens e controle da interface para usuário sem sessão.
profile.js / profile.css	Painel de perfil remoto; carrega e altera o nome de exibição em public.profiles; mostra nível e XP fornecidos pelo app.
supabase-config.js	URL do projeto Supabase e chave publicável. Não deve conter service_role, senha ou segredo OAuth.
supabase-profiles.sql	Estrutura esperada de public.profiles, trigger de criação e RPC de recuperação segura do perfil.
supabase-profiles-fix.sql	Correções idempotentes de trigger/perfil ausente para banco já existente.
supabase-profiles-verify.sql	Consultas de verificação da estrutura e do estado dos perfis.
sw.js	Service worker/PWA: cache de arquivos, atualização por CACHE_VERSION e estratégia de rede primeiro para código.
manifest.json e imagens	Metadados PWA, nome, ícones, mascote, logotipo e splash.
trail.css / auth.css / profile.css	Estilos separados das telas de trilha, autenticação e perfil.

Ordem de carregamento em index.html

O HTML carrega o SDK Supabase via CDN (versão fixada), supabase-config.js, auth.js, auth-ui.js, profile.js, core.js, program.js e trail.js. O SDK é solicitado com async; auth.js aguarda sua disponibilidade por um período limitado antes de concluir a inicialização. Ao adicionar um novo módulo, verificar a ordem de dependências e incluir o arquivo no cache do service worker quando adequado.

PWA e cache

O service worker atual registra CACHE_VERSION = 11. Arquivos principais ficam no cache; módulos relacionados à autenticação ficam como itens extras. Requisições ao domínio Supabase não são interceptadas nem armazenadas em cache. Toda publicação que alterar arquivos deve revisar e, quando necessário, incrementar CACHE_VERSION e atualizar CORE/EXTRA.

05 / AUTENTICAÇÃO

Supabase, contas e perfil remoto

O que já existe e o que ainda não significa sincronização do treino.

Autenticação implementada no código

- Cadastro com e-mail e senha; o nome informado é enviado como metadado full_name.
- Login com e-mail e senha.
- Login com Google por Supabase OAuth, usando a URL atual do app como retorno.
- Recuperação de senha e atualização de senha.
- Sessão persistida pelo SDK, renovação automática de token e detecção de retorno OAuth na URL.
- Logout local da sessão Supabase: signOut({ scope: "local" }). O código atual não apaga o estado local do Crowfit ao sair.

Perfil remoto

A tabela public.profiles é 1:1 com auth.users: a chave id referencia auth.users(id), com display_name, created_at e updated_at. Um trigger handle_new_user cria o perfil quando uma conta é criada. A RPC ensure_my_profile() recupera um perfil ausente usando auth.uid(), sem aceitar um ID enviado pelo navegador. O módulo profile.js confere que a linha retornada pertence ao UID atual e invalida carregamentos antigos quando ocorre troca de conta.

O QUE O PERFIL REMOTO NÃO GUARDA

No código SQL e no módulo profile.js consultados, public.profiles guarda apenas o nome de exibição e metadados de criação/atualização. Idade, peso, altura, experiência, dias de treino, pontos, ofensiva, sessões e trilha não são campos desse perfil remoto.

Segurança e limites

- A chave publicável do Supabase pode aparecer no frontend; service_role, senhas e Client Secret do Google nunca podem ser incluídos em arquivos públicos.
- RLS deve garantir que cada usuário consulte/edite apenas seus próprios registros. RPCs que atuem em dados do usuário devem usar auth.uid() e validar a sessão no servidor.
- O front-end concede pontos e recompensas. Isso é adequado para gamificação pessoal, mas não é um sistema antifraude para ranking competitivo, dinheiro ou prêmios.
- A presença das funções no código não comprova que o login Google está configurado corretamente no painel Supabase/Google nem que todos os fluxos foram testados em navegador real.

Nota sobre login Google

A implementação do botão Google está em auth.js e chama signInWithOAuth({ provider: "google" }). Para diagnóstico, verificar o UID efetivamente retornado pelo Supabase, provedores vinculados, URLs permitidas, callback do Supabase e comportamento após o redirect. Não inferir a causa de falhas de login apenas pelo e-mail exibido.

06 / DIAGNÓSTICO PRIORITÁRIO

Dados misturados entre contas

O comportamento relatado pelo usuário tem uma causa direta no modelo de armazenamento que aparece no repositório consultado.

Causa técnica confirmada no snapshot

index.html define uma única chave global de localStorage: corvofit-v1. A função load() lê essa chave antes de decidir qual tela mostrar. O estado carregado contém S.profile e todo o restante do progresso. No fim do carregamento, se S.profile existir, a aplicação abre a home; caso contrário, apresenta o onboarding.

Esse armazenamento não inclui o UID do usuário. Por isso, quando uma conta é trocada no mesmo navegador, a nova sessão Supabase pode ser diferente, mas o estado S já foi carregado da mesma chave local. O aplicativo encontra o perfil antigo e não pergunta novamente idade, peso, altura, experiência ou frequência de treino.

Por que o login não corrige isso sozinho

auth.js gerencia a sessão Supabase, mas declara que não lê nem grava as chaves de armazenamento do app. O logout encerra a sessão Supabase e não toca nos dados locais. profile.js tem controle próprio de UID e troca de conta para o perfil remoto, mas esse painel remoto não gerencia o objeto S nem os dados pessoais de treino. Assim, autenticação e estado de treino estão desacoplados.

RESUMO DO BUG

A sessão muda; a chave corvofit-v1 não muda. O aplicativo carrega a mesma estrutura local para qualquer usuário que use aquele navegador. Isso explica a idade, o peso e o plano antigos aparecerem após trocar de conta.

Direção correta da correção

1. Identificar o usuário pelo ID único (UID) do Supabase, nunca pelo e-mail como chave primária.
2. Separar os dados locais por usuário (por exemplo, um namespace por UID) ou associá-los a um repositório de dados com contexto de conta explícito.
3. Só carregar a partição de dados depois de a identidade da sessão estar resolvida. Não decidir onboarding/home antes de saber a conta ativa.
4. Ao trocar de conta ou sair, invalidar tarefas assíncronas e limpar o estado em memória da conta anterior, sem apagar a partição persistida daquela conta.
5. Conta nova sem perfil de treino deve abrir o onboarding. Conta existente deve recuperar somente seus próprios dados.
6. Migrar corvofit-v1 uma única vez, com cópia de segurança e regra explícita de adoção. Nunca copiar os dados antigos automaticamente para toda conta que entrar no dispositivo.

Estratégia de migração sugerida

Os dados atuais sem UID são “legados sem proprietário identificado”. A adoção deve ser feita uma vez, com estado de migração persistente e uma decisão clara sobre a primeira conta autorizada a recebê-los. Se a conta já possui progresso remoto, não sobrescrever com o legado sem uma política de conflito clara. Preservar uma cópia imutável do legado até a validação final.

07 / PRÓXIMAS MUDANÇAS AUTORIZADAS

Escopo atual definido pelo usuário

Duas melhorias em andamento. Não ampliar o escopo sem autorização explícita.

A. Isolamento de dados por conta

- Cada conta deve ter seu próprio perfil de treino e seu próprio progresso.
- A conta nova deve passar pelo onboarding se ainda não tiver perfil de treino.
- Ao retornar à conta antiga, recuperar somente os dados dela.
- Trocar de conta não pode apagar dados de outras contas nem fazer merge entre elas.
- A decisão anterior de produto foi manter idade, peso e altura localmente, enquanto pontos/nível/ofensiva/plano/histórico poderiam ser sincronizados. Até nova autorização, preservar essa distinção; se os dados pessoais ficarem locais, isolá-los por UID.
- Preferência anterior de migração: dados locais antigos podem ser adotados uma única vez por uma primeira conta elegível; contas existentes devem restaurar seus próprios dados de nuvem, sem mistura.

B. Reorganizar as configurações

O usuário quer retirar “Backup e perfil” da página principal do treino e colocar backup dentro de uma tela de configurações organizada por categorias. A tela inicial de treino deve ficar mais limpa.

CATEGORIA	CONTEÚDO PRETENDIDO
Conta e perfil	Nome de exibição, informações da conta autenticada e sair da conta.
Perfil de treino	Nome, idade, peso, altura, experiência, dias por semana e ação para refazer o perfil/plano.
Dados e backup	Exportar backup, importar backup, explicar o que o arquivo inclui e confirmar antes de substituir dados.

- A entrada para configurações deve ficar na área de perfil/conta, com botão ou ícone de engrenagem.
- Cada categoria abre sua própria seção e mostra apenas os controles correspondentes; incluir navegação de volta.
- Manter identidade visual Crowfit, responsividade Android/iOS e acessibilidade básica.
- Não criar opções fictícias nem controles sem funcionalidade.

RESTRIÇÃO DE ESCOPO

Nesta etapa, trabalhar apenas no isolamento de dados por conta e na organização da área de configurações. Não adicionar conquistas, ranking, avatar, lembretes, novos treinos ou outras funcionalidades sem pedido.

Decisões anteriores a preservar

- Não exigir nome de usuário @ único; display_name é suficiente por enquanto.
- Manter o mascote corvo existente; não redesenhar ou recolorir o PNG em alterações de interface.
- Avatares podem ser variações fixas do corvo; desbloqueios são possibilidade futura, não escopo atual.

08 / PROTOCOLO DE DESENVOLVIMENTO

Como a IA deve trabalhar no Crowfit

Orientações para Claude, ChatGPT ou qualquer agente que altere o repositório.

Antes de modificar

- Ler o repositório completo e identificar a branch, commit e arquivos realmente existentes. Não assumir que o PDF descreve a versão mais nova.
- Inspecionar os arquivos diretamente relacionados ao pedido e seguir as chamadas entre módulos. README e documentação podem estar desatualizados.
- Mapear dados que podem ser alterados, onde são persistidos, quem os consome e como a migração será revertida.
- Se houver SQL novo, ler o arquivo integral, conferir dependências, tabelas, colunas, RPCs, RLS, grants e impacto sobre o banco já existente antes de sugerir execução.

Durante a implementação

- Fazer mudanças mínimas e localizadas. Não reescrever o app inteiro nem trocar o stack vanilla HTML/CSS/JS sem necessidade.
- Preservar IDs de exercícios, formatos de backup e chaves de progresso sempre que possível; se mudar formato, criar migração versionada.
- Não apagar dados, tabelas, políticas ou funções sem justificar, prever backup e apresentar plano de rollback.
- Nunca colocar segredos no frontend ou em arquivos públicos do GitHub.
- Garantir isolamento por UID, RLS no banco e validação de propriedade. Não confiar apenas em filtros de interface.
- Ao criar novo JS/CSS, atualizar ordem de scripts e service worker (CORE/EXTRA e CACHE_VERSION) quando necessário.
- Evitar logs de tokens, credenciais, peso, idade e demais dados pessoais.

Depois da implementação

- Rodar verificações de sintaxe e testes relevantes; não afirmar que testou no navegador/Supabase se não executou de fato.
- Testar duas contas diferentes no mesmo aparelho, uma conta nova, uma conta antiga, troca de conta, logout, login por e-mail, Google e retorno de OAuth.
- Testar migração do legado, preservação dos backups, importação/exportação, perfil e trilha após atualizar o app e em modo offline.
- Reportar arquivos alterados, mudanças no banco, SQL a executar, testes feitos/resultados, limitações e instruções de publicação.

FORMATO DE ENTREGA ESPERADO

Ao concluir cada tarefa, apresentar: (1) resumo da mudança; (2) arquivos modificados; (3) SQL/migrações necessários e em que ambiente aplicar; (4) testes realizados e resultados reais; (5) riscos, pendências e rollback. Não dizer “concluído” quando apenas o código foi escrito, mas ainda não foi testado ou publicado.

09 / REFERÊNCIA E LIMITES

Estado verificado e pontos em aberto

Este retrato técnico corresponde ao conteúdo público observado em 10 de novembro de 2026.

O que foi confirmado no GitHub público

- A branch main apresenta os módulos index.html, core.js, program.js, trail.js, auth.js, auth-ui.js e profile.js, além dos arquivos de estilo, PWA e SQL de perfil.
- O estado principal continua sendo salvo em localStorage na chave única corvofit-v1; load() e save() não recebem UID.
- A tabela/SQL de perfis visível cobre perfil remoto simples (display_name), não a totalidade do perfil de treino ou da trilha.
- O README ainda descreve limitações antigas: progresso local, ausência de sincronização entre aparelhos e perfil remoto ainda não implementado. Isso diverge da existência atual de profile.js e dos SQLs de perfil.
- No root da branch main que foi consultada, não foi possível localizar supabase-sync.sql nem um módulo JavaScript dedicado à sincronização do progresso. Pode existir em outra branch, caminho, alteração ainda não enviada ou versão mais recente; conferir antes de qualquer conclusão definitiva.
- O service worker está em CACHE_VERSION 11 no snapshot consultado.

Pontos que ainda precisam de confirmação

- Qual é o caminho/branch/commit do SQL de sincronização mais recente mencionado pelo usuário e se ele foi realmente aplicado no projeto Supabase atual.
- Se existe um módulo de sincronização de progresso em outra branch ou se o código ainda não chama as RPCs do SQL.
- Quais campos exatamente devem ser sincronizados para nuvem e quais devem permanecer apenas no dispositivo, especialmente idade, peso e altura.
- Como o Supabase/Google está configurado no painel e se o fluxo OAuth passou por teste prático após as mudanças recentes.
- Se a versão publicada no GitHub Pages corresponde ao commit mais recente da branch main e se o service worker do aparelho já atualizou o cache.

Fontes técnicas consultadas

Repositório e lista de arquivos: https://github.com/virusdosol/crowfit
README: https://raw.githubusercontent.com/virusdosol/crowfit/main/README.md
index.html: https://raw.githubusercontent.com/virusdosol/crowfit/main/index.html
core.js: https://raw.githubusercontent.com/virusdosol/crowfit/main/core.js
program.js: https://raw.githubusercontent.com/virusdosol/crowfit/main/program.js
trail.js: https://raw.githubusercontent.com/virusdosol/crowfit/main/trail.js
auth.js: https://raw.githubusercontent.com/virusdosol/crowfit/main/auth.js
auth-ui.js: https://raw.githubusercontent.com/virusdosol/crowfit/main/auth-ui.js
profile.js: https://raw.githubusercontent.com/virusdosol/crowfit/main/profile.js
SQL de perfis: https://raw.githubusercontent.com/virusdosol/crowfit/main/supabase-profiles.sql
Correção do SQL de perfis: https://raw.githubusercontent.com/virusdosol/crowfit/main/supabase-profiles-fix.sql
Service worker: https://raw.githubusercontent.com/virusdosol/crowfit/main/sw.js

LEMBRETE PARA TODA NOVA TAREFA

Anexe este documento e escreva no prompt: “Use o PDF como contexto do produto e das decisões já tomadas. Primeiro confira o estado atual do repositório e identifique qualquer divergência. Implemente somente o escopo descrito neste prompt, preservando os dados e as funcionalidades existentes.”